1 Preliminaries

In this assignment, you will practice working with the kinematics of the 7DOF Kuka iiwa robot. You will perform position control using numerical IK, and you will also experiment with P and PI controllers for velocity inputs.

[1]:
```
# Run once at the beginning to install pybullet
!pip install pybullet
```

```
Requirement already satisfied: pybullet in
c:\users\micha\anaconda3\envs\cv\lib\site-packages (3.2.5)
```

[2]:
```
import pybullet as p
import pybullet_data
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import animation
```

[3]:
```
# some utility functions

def plot_joint_progression(solution):
    """Plots the joint angle progression of a given set of joint configurations.
 ↪"""
    joint_configs_array = np.array(solution)
    plt.figure(figsize=(8, 6))
    for i in range(num_joints):
        plt.plot(joint_configs_array[:, i], label=f'Joint {i+1}')

    plt.title('Joint Angle Progression')
    plt.xlabel('Iteration')
    plt.ylabel('Joint Angle (radians)')
    plt.legend()
    plt.grid(True)
    plt.show()


plt.rcParams["animation.html"] = "jshtml"
plt.rcParams['figure.dpi'] = 90
plt.ioff()

def animate_solution(path):
    fig, ax = plt.subplots()
    def animate(frame):
        ax.cla()
        for i in range(len(path[frame])):
            p.resetJointState(robot_id, i, path[frame][i],␣
 ↪physicsClientId=sim_id)
        showImage(ax)
```

```
    anim = animation.FuncAnimation(fig, animate, frames=len(path))
    plt.close(fig)
    return anim
```

[4]:
```
sim_id = p.connect(p.DIRECT)
p.setAdditionalSearchPath(pybullet_data.getDataPath())

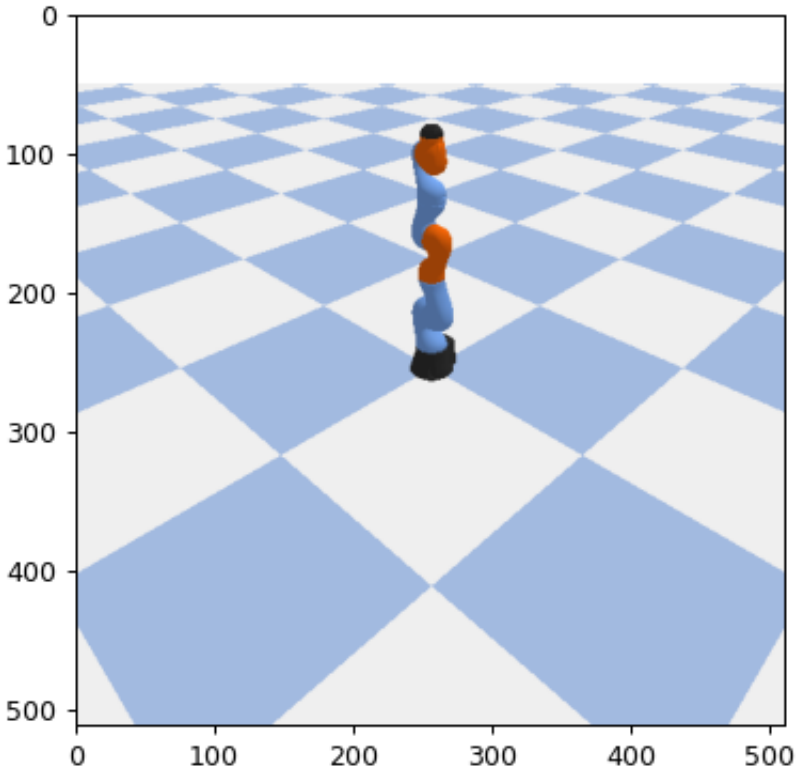
plane_id = p.loadURDF('plane.urdf', physicsClientId=sim_id)

robot_id = p.loadURDF("kuka_iiwa/model.urdf",
             basePosition=[0,0,0],
             baseOrientation=p.getQuaternionFromEuler([0,0,0]),
             useFixedBase=True,
             globalScaling=1,
             flags=p.URDF_USE_IMPLICIT_CYLINDER,
             physicsClientId=sim_id)

num_joints = p.getNumJoints(robot_id, physicsClientId=sim_id)
```

[5]:
```
def showImage(ax, cameraPos=[2, 2, 2]):
  viewMatrix = p.computeViewMatrix(
               cameraEyePosition=cameraPos,
               cameraTargetPosition=[0, 0, 0],
               cameraUpVector=[0, 0, 1],
               physicsClientId=sim_id)
  projectionMatrix = p.computeProjectionMatrixFOV(
               fov=60.0,
               aspect=1.0,
               nearVal=0.1,
               farVal=10,
               physicsClientId=sim_id)
  width, height, rgbImg, depthImg, segImg = p.getCameraImage(
               width=512,
               height=512,
               viewMatrix=viewMatrix,
               projectionMatrix=projectionMatrix,
               physicsClientId=sim_id)
  ax.imshow(rgbImg)

fig, ax = plt.subplots()
showImage(ax)
```



2 Part 1: Forward Kinematics

For a pre-loaded robot model, Pybullet is already able to compute the forward kinematics of each link and the end effector. Specifically, `p.getLinkState(robot_id, link_id, computeForwardKinematics=1, sim_id)` returns a tuple, where the first element is the link position and the second element is the link orientation in quaternion form, both relative to the world frame.

2.1 1.1: Absolute Transformations (6 pts)

Extract and store the absolute homogeneous transformations T_i^0 for all frames $i \in \{1, ..., 7\}$. Pybullet uses 0-indexing here, so the first joint frame has index 0 and the end effector frame has index 6.

Recall that you can convert the quaternion into a rotation matrix using `p.getMatrixFromQuaternion()`.

[6]:
```
absolute_transforms = []
for i in range(num_joints):
    state = p.getLinkState(robot_id, i, computeForwardKinematics=1,
                           physicsClientId=sim_id)
    T = np.eye(4)
```

```
    T[:3, :3] = np.array(p.getMatrixFromQuaternion(state[1])).reshape(3, 3)
    T[:3, 3] = state[0]
    absolute_transforms.append(T)
```

2.2 1.2: Relative Transformations (6 pts)

Given the *absolute* transformations above, we can then compute the *relative* transformation matrices T_i^{i-1} as follows:

$$T_i^{i-1} = (T_{i-1}^0)^{-1} T_i^0$$

Compute each matrix T_i^{i-1} and print out them out. Also separately print out the corresponding translation and rotation, the latter as a set of xyz Euler angles.

To obtain the latter from a rotation matrix, you can use `scipy.spatial.transform.Rotation.from_matrix(rotation_matrix).as_euler('xyz', degrees=True)`.

[7]:
```
from scipy.spatial.transform import Rotation as R
import warnings

relative_transforms = []
previous_T = np.eye(4)
for i, T in enumerate(absolute_transforms, start=1):
    relative_T = np.linalg.inv(previous_T) @ T
    relative_transforms.append(relative_T)
    with warnings.catch_warnings():
        warnings.filterwarnings('ignore', message='Gimbal lock detected')
        angles = R.from_matrix(relative_T[:3, :3]).as_euler('xyz', degrees=True)
    print(f'T^{i-1}_{i}:')
    print(np.round(relative_T, 5))
    print('Translation:', np.round(relative_T[:3, 3], 5))
    print('xyz Euler angles (degrees):', np.round(angles, 5))
    print()
    previous_T = T
```

```
T^0_1:
[[ 1.      0.      0.      0.    ]
 [ 0.      1.      0.     -0.03  ]
 [ 0.      0.      1.      0.2775]
 [ 0.      0.      0.      1.    ]]
Translation: [ 0.     -0.03    0.2775]
xyz Euler angles (degrees): [0. 0. 0.]

T^1_2:
[[-1.000e+00  0.000e+00 -0.000e+00 -3.000e-04]
 [-0.000e+00 -0.000e+00  1.000e+00  7.200e-02]
 [ 0.000e+00  1.000e+00  0.000e+00  1.415e-01]
```

```
 [ 0.000e+00  0.000e+00  0.000e+00  1.000e+00]]
Translation: [-0.0003  0.072   0.1415]
xyz Euler angles (degrees): [  90.    0. -180.]

T^2_3:
[[-1.000e+00  0.000e+00 -0.000e+00 -3.000e-04]
 [-0.000e+00 -0.000e+00  1.000e+00  2.755e-01]
 [ 0.000e+00  1.000e+00  0.000e+00 -1.200e-02]
 [ 0.000e+00  0.000e+00  0.000e+00  1.000e+00]]
Translation: [-0.0003  0.2755 -0.012 ]
xyz Euler angles (degrees): [  90.    0. -180.]

T^3_4:
[[ 1.      0.     -0.      0.    ]
 [-0.      0.     -1.     -0.064 ]
 [-0.      1.      0.      0.1525]
 [ 0.      0.      0.      1.    ]]
Translation: [ 0.     -0.064   0.1525]
xyz Euler angles (degrees): [90.  0.  0.]

T^4_5:
[[-1.000e+00  0.000e+00 -0.000e+00 -1.000e-04]
 [-0.000e+00  0.000e+00  1.000e+00  1.935e-01]
 [ 0.000e+00  1.000e+00 -0.000e+00 -1.300e-02]
 [ 0.000e+00  0.000e+00  0.000e+00  1.000e+00]]
Translation: [-1.000e-04  1.935e-01 -1.300e-02]
xyz Euler angles (degrees): [ 90.  -0. 180.]

T^5_6:
[[ 1.000e+00 -0.000e+00 -0.000e+00 -1.000e-04]
 [-0.000e+00  0.000e+00 -1.000e+00 -2.140e-02]
 [ 0.000e+00  1.000e+00  0.000e+00  1.401e-01]
 [ 0.000e+00  0.000e+00  0.000e+00  1.000e+00]]
Translation: [-1.000e-04 -2.140e-02  1.401e-01]
xyz Euler angles (degrees): [90.  0.  0.]

T^6_7:
[[-1.000e+00  0.000e+00 -0.000e+00 -0.000e+00]
 [-0.000e+00  0.000e+00  1.000e+00  1.004e-01]
 [ 0.000e+00  1.000e+00 -0.000e+00 -4.000e-04]
 [ 0.000e+00  0.000e+00  0.000e+00  1.000e+00]]
Translation: [-0.      0.1004 -0.0004]
xyz Euler angles (degrees): [ 90.  -0. 180.]
```

3 Part 2: Jacobian and Numerical IK

Now we will investigate the robot's Jacobian and use it to solve the inverse kinematics problem. We can retrieve the robot's instantaneous numerical Jacobian by calling `p.calculateJacobian()` with the usage shown in the following code cell:

[8]:
```
q = [p.getJointState(robot_id, i)[0] for i in range(num_joints)]

linear_jacobian, angular_jacobian = p.calculateJacobian(
    robot_id,
    linkIndex=num_joints-1, # the link for which we want the Jacobian
    localPosition=[0,0,0],  # the point, relative to the local frame, for which␣
 ↪we want the Jacobian
    objPositions=q,         # robot's joint configurations
    objVelocities=[0]*num_joints,    # Joint velocity list, usually 0 for␣
 ↪kinematics
    objAccelerations=[0]*num_joints, # Joint acceleration list, usually 0
    physicsClientId=sim_id
)
```

This returns two Jacobian matrices, one corresponding to linear velocities and the other two angular velocities. We will only be concerned with the former in this assignment.

3.1 2.1: Jacobian and Singularity Analysis (6 pts)

Complete the following tasks and answer the corresponding questions.

1. Use `p.resetJointState()` to set all joint angles to $\mathbf{q} = \mathbf{0}$ and compute the linear velocity Jacobian. Convert it into matrix form, round it to several decimal places, and print it.
2. What is the rank of the Jacobian? Which joints are able to produce nonzero motion at the end effector? Along which directions or in what space does each of those motions lie?
3. Repeat the above analysis for the configuration $\mathbf{q} = (0, 0, 0, \pi/2, 0, 0, 0)$.

[9]:
```
for i in range(num_joints):
    p.resetJointState(robot_id, i, 0, physicsClientId=sim_id)
q = [p.getJointState(robot_id, i, physicsClientId=sim_id)[0]
     for i in range(num_joints)]
J_zero = np.array(p.calculateJacobian(
    robot_id, num_joints - 1, [0, 0, 0], q,
    [0] * num_joints, [0] * num_joints,
    physicsClientId=sim_id)[0])
print('Linear Jacobian at q = 0:')
print(np.round(J_zero, 5))
print('Rank:', np.linalg.matrix_rank(J_zero, tol=1e-8))
print('Nonzero columns (joints numbered 1-7):',
      np.flatnonzero(np.linalg.norm(J_zero, axis=0) > 1e-8) + 1)
```

```
Linear Jacobian at q = 0:
[[-0.     0.901 -0.    -0.481  0.     0.081  0.   ]
```

```
 [ 0.     0.    -0.    -0.     0.     0.     0.   ]
 [ 0.    -0.     0.     0.    -0.    -0.     0.   ]]
Rank: 1
Nonzero columns (joints numbered 1-7): [2 4 6]
```

[10]:
```
q_bent = np.array([0, 0, 0, np.pi / 2, 0, 0, 0])
for i in range(num_joints):
    p.resetJointState(robot_id, i, q_bent[i], physicsClientId=sim_id)
J_bent = np.array(p.calculateJacobian(
    robot_id, num_joints - 1, [0, 0, 0], q_bent.tolist(),
    [0] * num_joints, [0] * num_joints,
    physicsClientId=sim_id)[0])
print('Linear Jacobian at q = (0, 0, 0, pi/2, 0, 0, 0):')
print(np.round(J_bent, 5))
print('Rank:', np.linalg.matrix_rank(J_bent, tol=1e-8))
print('Nonzero columns (joints numbered 1-7):',
      np.flatnonzero(np.linalg.norm(J_bent, axis=0) > 1e-8) + 1)
```

```
Linear Jacobian at q = (0, 0, 0, pi/2, 0, 0, 0):
[[ 0.     0.42  -0.    -0.     0.     0.     0.   ]
 [-0.481 -0.    -0.481 -0.     0.     0.     0.   ]
 [-0.     0.481 -0.    -0.481  0.     0.081  0.   ]]
Rank: 3
Nonzero columns (joints numbered 1-7): [1 2 3 4 6]
```

3.1.1 Answer to 2.1

Joint numbers below are 1-based (PyBullet indices are one less). At the zero configuration, the linear Jacobian has rank 1. Only joints 2, 4, and 6 have nonzero columns, all along the world x-axis: their velocities are approximately (0.901, 0, 0), (-0.481, 0, 0), and (0.081, 0, 0) per unit joint velocity. Thus instantaneous end-effector translation is restricted to a line. Joints 1, 3, 5, and 7 produce no instantaneous translation at this point, although they can affect orientation.

At q = (0, 0, 0, pi/2, 0, 0, 0), the rank is 3. Joints 1 and 3 move the end effector along the negative y-axis; joint 2 produces a direction in the x-z plane, approximately (0.420, 0, 0.481); joint 4 acts along negative z; and joint 6 acts along positive z. Joints 5 and 7 have zero linear columns. Together the nonzero columns span three-dimensional translation, so this configuration is nonsingular for the position task. Near-zero numerical entries are treated as zero using a tolerance of 1e-8.

3.2 2.2: Numerical Inverse Kinematics (8 pts)

Now that we have an understanding of the Jacobian, we will implement the Newton-Raphson method with damping for numerical IK. Complete the `numerical_ik()` function below. Given a `target_pos` for the end effector, it should run interatively for at most `max_iter` iterations or until the error becomes smaller than `tol`, whichever occurs first.

In each iteration, find the current end effector position, Jacobian, and position error. Compute the *damped* Jacobian pseudoinverse as $J^\top(JJ^\top + \lambda^2 I)^{-1}$, where λ^2 is provided as `lam2`. Apply the corresponding update to the configuration with step size `step`, and reset the robot to the new configuration.

Finally, your function should store a list of the updated configuration in each iteration. Return this list when the function finishes.

[11]:
```
def numerical_ik(target_pos, lam2, step=0.1, max_iter=100, tol=1e-3):
    """
    Solves IK for target position using damped Jacobian Pseudo-Inverse.
    Updates the simulated robot's configuration at each iteration of the IK␣
 ↪search.
    Returns a list of joint configurations computed in each step.
    """
    target_pos = np.asarray(target_pos, dtype=float)
    if lam2 < 0:
        raise ValueError('lam2 must be nonnegative')
    solution = []
    for _ in range(max_iter):
        q = np.array([p.getJointState(robot_id, i, physicsClientId=sim_id)[0]
                      for i in range(num_joints)])
        position = np.array(p.getLinkState(
            robot_id, num_joints - 1, computeForwardKinematics=1,
            physicsClientId=sim_id)[0])
        error = target_pos - position
        if np.linalg.norm(error) < tol:
            break
        J = np.array(p.calculateJacobian(
            robot_id, num_joints - 1, [0, 0, 0], q.tolist(),
            [0] * num_joints, [0] * num_joints,
            physicsClientId=sim_id)[0])
        if lam2 == 0:
            J_inverse = np.linalg.pinv(J, rcond=1e-8)
        else:
            J_inverse = J.T @ np.linalg.solve(
                J @ J.T + lam2 * np.eye(3), np.eye(3))
        q = q + step * (J_inverse @ error)
        for i in range(num_joints):
            p.resetJointState(robot_id, i, q[i], physicsClientId=sim_id)
        solution.append(q.copy())
    return solution
```

3.3 2.3 Testing IK (4 pts)

The following method can be invoked to test your implementation. The robot starts at the zero configuration, and we specify a workspace target. We run IK with a given damping parameter, plot the joint angle trajectories, and verify the position reached by the end effector. Finally, we plot the robot's joints over time, and we also return the joint trajectories as a list.

[12]:
```
def test_ik(lam2):
    for i in range(num_joints):
        p.resetJointState(robot_id, i, 0, physicsClientId=sim_id)
```

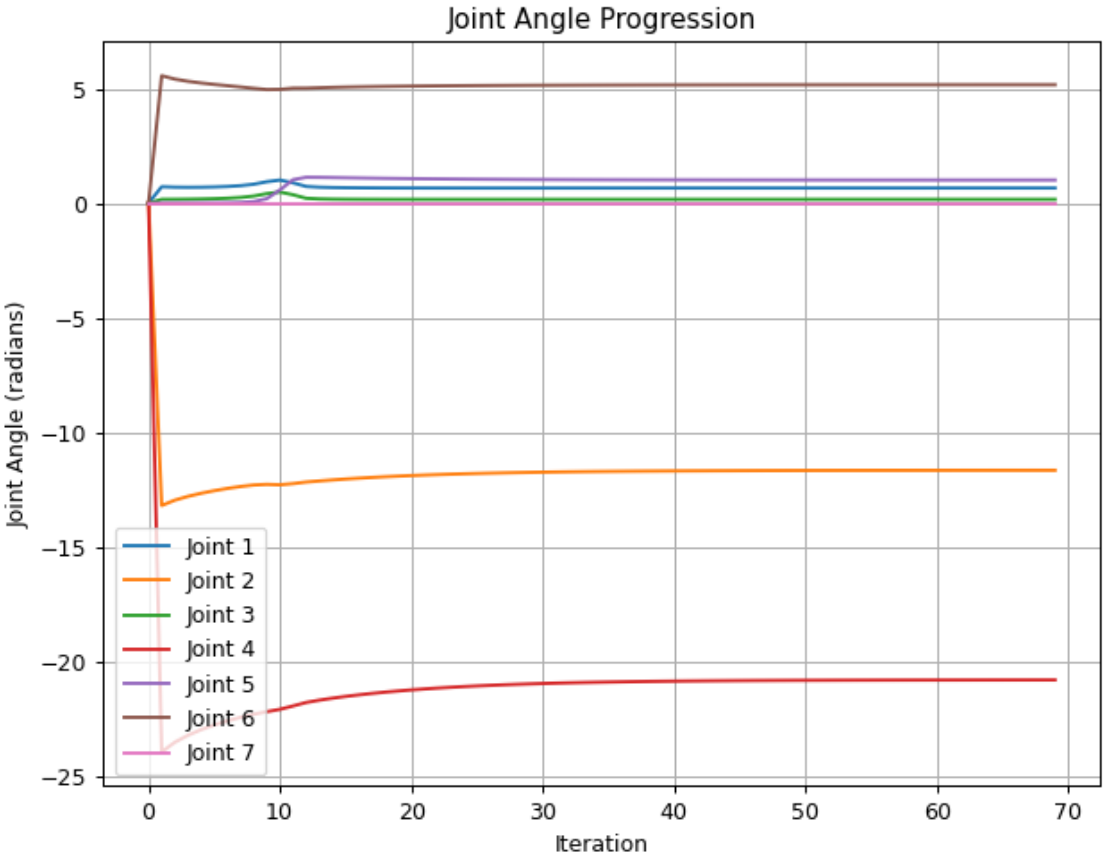
```
    target = np.array([0.4, 0.3, 0.2])
    solution = numerical_ik(target, lam2)
    actual_pos = p.getLinkState(robot_id, 6, computeForwardKinematics=1,␣
↪physicsClientId=sim_id)[0]
    print(f"Target: {np.round(target,3)}, Reached: {np.round(actual_pos,3)}")

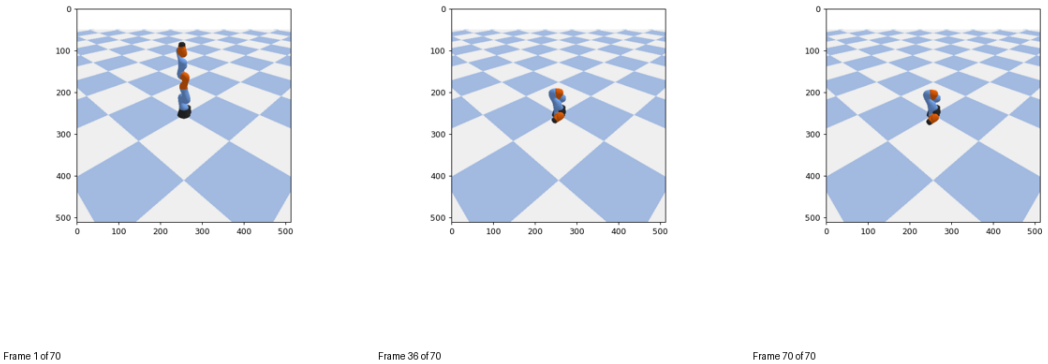
    plot_joint_progression(solution)
    return solution
```

Run `test_ik()` twice, first with $\lambda^2 = 0$ then with $\lambda^2 = 0.1$. Show both plots and use `animate_solution()` to visualize the robot's movement. (You can generate them in separate cells. Make sure that the animations do not cover the plots in your submission.) Briefly compare the two results and give an explanation for your observations, particularly with regard to the impact of singularities.

[13]:
```
from IPython.display import display
plt.rcParams['animation.embed_limit'] = 100
solution_ik_zero = test_ik(0.0)
ik_zero_reached = np.array(p.getLinkState(
    robot_id, 6, computeForwardKinematics=1, physicsClientId=sim_id)[0])
animation_ik_zero = animate_solution(solution_ik_zero)
display(animation_ik_zero)
```

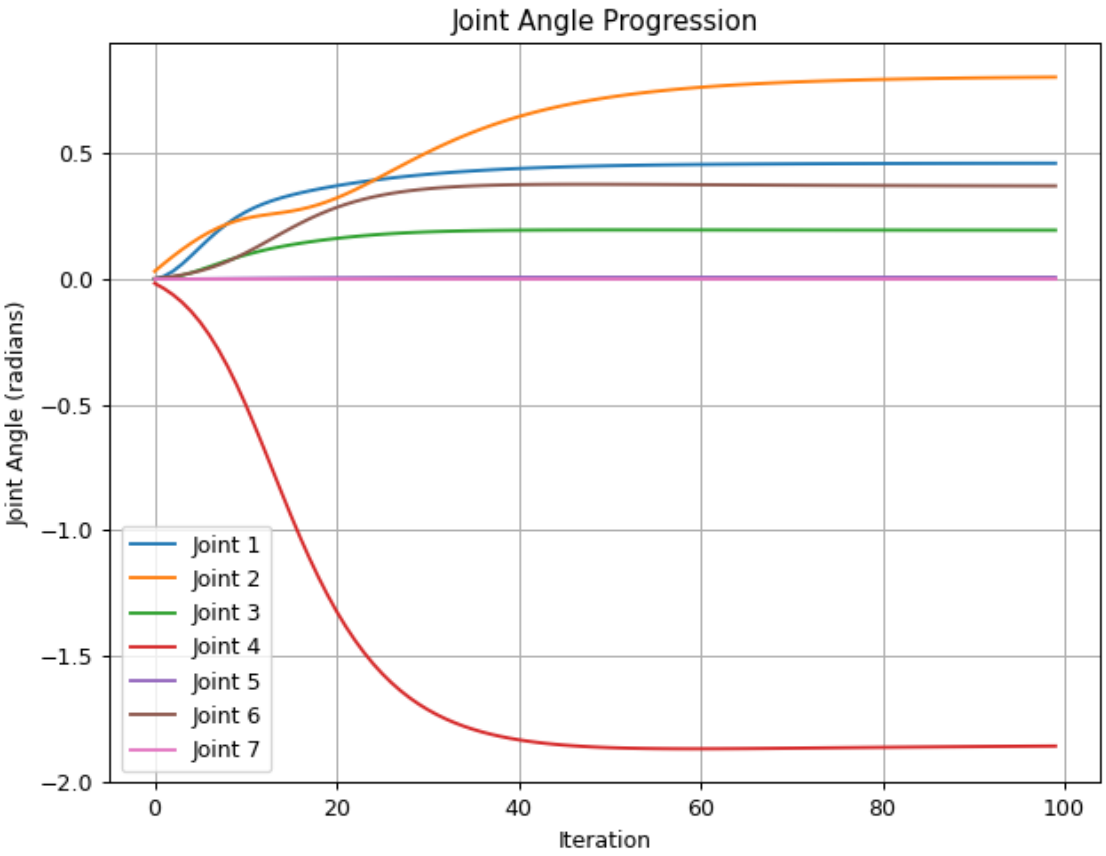
```
Target: [0.4 0.3 0.2], Reached: [0.399 0.299 0.2  ]
```

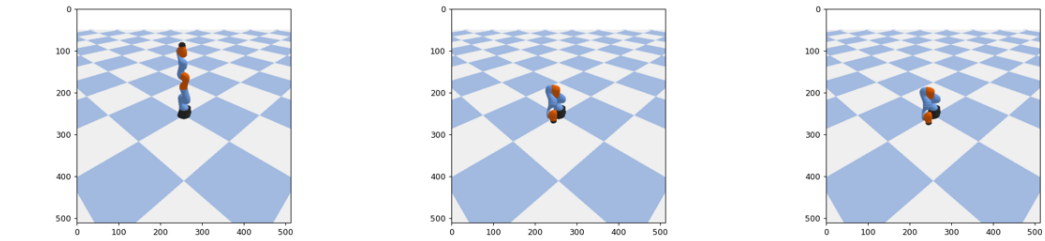




[14]:
```
solution_ik_damped = test_ik(0.1)
ik_damped_reached = np.array(p.getLinkState(
    robot_id, 6, computeForwardKinematics=1, physicsClientId=sim_id)[0])
animation_ik_damped = animate_solution(solution_ik_damped)
display(animation_ik_damped)
```

```
Target: [0.4 0.3 0.2], Reached: [0.399 0.299 0.202]
```





Frame 1 of 100 Frame 51 of 100 Frame 100 of 100

3.3.1 Answer to 2.3

With lambda squared = 0, IK reaches an error of about 0.000919 m in 70 updates. However, the largest joint update is about 27.93 radians, visible as a sharp jump in the trajectories. Small

nonzero singular values amplify the update, and joint angles can leave physical limits because this position-only IK exercise does not impose joint limits. At the exactly singular start, the ordinary inverse of J J transpose does not exist. The implementation uses the Moore-Penrose pseudoinverse for this zero-damping case, discarding numerically zero singular values rather than attempting an undefined inverse.

With lambda squared = 0.1, the largest update is about 0.0942 radians and the trajectories are smoother. Damping replaces singular-value amplification 1/sigma with sigma/(sigma squared + lambda squared), suppressing large updates near singularities. The resulting motion is more stable but slower: after the prescribed 100 updates the error is about 0.00206 m, so the 0.001 m tolerance has not yet been reached.

4 Part 3: Feedback Control

For real robots, we typically do not set joint positions directly as we have been doing with `p.resetJointState()`. Instead, we command velocities or torques. To command velocities, we can call `p.setJointMotorControlArray()` as shown in the following cell:

[15]:
```
dq = np.zeros(num_joints)

p.setJointMotorControlArray(
    robot_id,
    jointIndices=range(num_joints),    # which joints to command (can be some or␣
 ↪all)
    controlMode=p.VELOCITY_CONTROL,    # alternatively, p.TORQUE_CONTROL
    targetVelocities=dq,               # list of target velocities
    physicsClientId=sim_id
)
```

4.1 3.1: P Controller (8 pts)

Write a simple P controller for velocity inputs to the robot in order to achieve a fixed target configuration. As with numerical IK, this will be implemented in discrete steps.

In each iteration, retrieve the current configuration and compute the position error. Then apply the control law to compute the desired velocity. Remember that for a multi-joint robot, we have a gain matrix, proportional to the identty matrix.

Command the velocities to the robot joints, and call `p.stepSimulation(sim_id)` to effect physical movement before ending the current iteration. Finally, as with numerical IK, your function should store a list of the updated configuration in each iteration. Return this list when the function finishes.

[16]:
```
def p_control(target_q, Kp, max_iter=500, tol=1e-3):
    """
    P joint position controller given fixed target.
    Returns a list of joint configurations computed in each step.
    """
    target_q = np.asarray(target_q, dtype=float)
```

```
    gain = Kp * np.eye(num_joints)
    solution = []
    for _ in range(max_iter):
        q = np.array([p.getJointState(robot_id, i, physicsClientId=sim_id)[0]
                      for i in range(num_joints)])
        error = target_q - q
        if np.linalg.norm(error) < tol:
            break
        dq = gain @ error
        p.setJointMotorControlArray(
            robot_id, range(num_joints), p.VELOCITY_CONTROL,
            targetVelocities=dq.tolist(), physicsClientId=sim_id)
        p.stepSimulation(physicsClientId=sim_id)
        q = np.array([p.getJointState(robot_id, i, physicsClientId=sim_id)[0]
                      for i in range(num_joints)])
        solution.append(q.copy())
    p.setJointMotorControlArray(
        robot_id, range(num_joints), p.VELOCITY_CONTROL,
        targetVelocities=[0] * num_joints, physicsClientId=sim_id)
    return solution
```

Run the following code cell to test your implementation. The robot starts at the zero configuration, and we specify a target configuration. You can again verify that the animation produces expected robot motion.

[17]:
```
for i in range(num_joints):
    p.resetJointState(robot_id, i, 0, physicsClientId=sim_id)

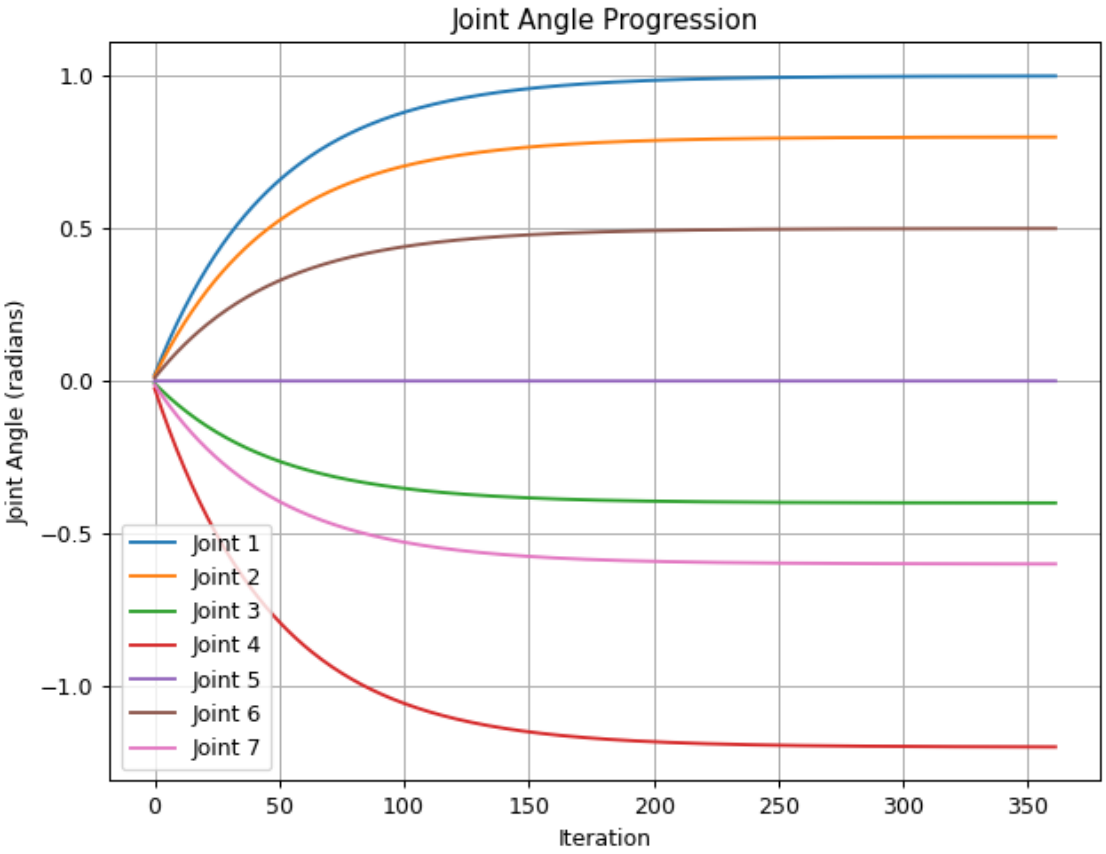
Kp = 5.0
target_q = np.array([1.0, 0.8, -0.4, -1.2, 0.0, 0.5, -0.6])
solution = p_control(target_q, Kp)

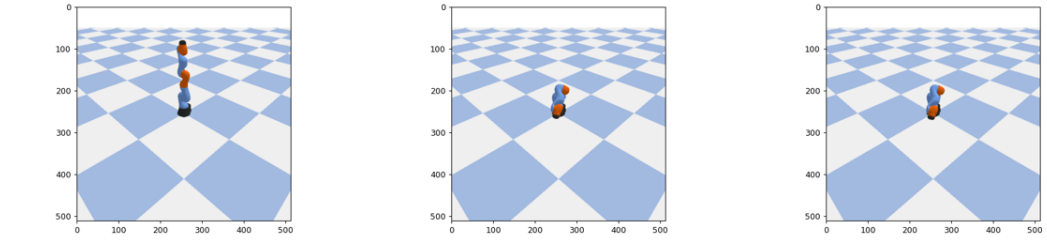
final_states = [p.getJointState(robot_id, i, sim_id) for i in range(num_joints)]
final_q_reached = np.array([s[0] for s in final_states])

print(f"Final q (target): {np.round(target_q, 3)}")
print(f"Final q (reached): {np.round(final_q_reached, 3)}")
print(f"Final position error norm: {np.linalg.norm(target_q - final_q_reached):.
 ↪4f}")

plot_joint_progression(solution)
animate_solution(solution)
```

```
Final q (target): [ 1.   0.8 -0.4 -1.2  0.   0.5 -0.6]
Final q (reached): [ 1.     0.8   -0.4   -1.199  0.     0.5   -0.6  ]
Final position error norm: 0.0010
```





Frame 1 of 362 Frame 182 of 362 Frame 362 of 362

4.2 3.2: PI Controller (8 pts)

Now let's suppose we want to track a moving target configuration. A PI controller is typically preferred for this task, since the integral term can better handle steady-state error.

Implement this controller in the function below. It should share a similar structure to the P

controller, with a few key differences:

- `target_q` is a list of target positions.
- The number of steps to run the controller is equal to the length of `target_q`.
- The "integral" is updated discretely in each step by an increment of `q_error * dt`.

The value of `dt` is 240 Hz, because each call of `p.stepSimulation()` advances time by $\frac{1}{240}$ seconds. (It is also possible to change this in Pybullet, but we will not worry about it here.)

[18]:
```
def pi_control(target_q, Kp, Ki, dt=1.0/240.0):
    """
    PI joint position controller given list of target configurations.
    Returns a list of joint configurations computed in each step.
    """
    targets = np.asarray(target_q, dtype=float)
    proportional_gain = Kp * np.eye(num_joints)
    integral_gain = Ki * np.eye(num_joints)
    integral = np.zeros(num_joints)
    solution = []
    for target in targets:
        q = np.array([p.getJointState(robot_id, i, physicsClientId=sim_id)[0]
                      for i in range(num_joints)])
        error = target - q
        integral += error * dt
        dq = proportional_gain @ error + integral_gain @ integral
        p.setJointMotorControlArray(
            robot_id, range(num_joints), p.VELOCITY_CONTROL,
            targetVelocities=dq.tolist(), physicsClientId=sim_id)
        p.stepSimulation(physicsClientId=sim_id)
        q = np.array([p.getJointState(robot_id, i, physicsClientId=sim_id)[0]
                      for i in range(num_joints)])
        solution.append(q.copy())
    p.setJointMotorControlArray(
        robot_id, range(num_joints), p.VELOCITY_CONTROL,
        targetVelocities=[0] * num_joints, physicsClientId=sim_id)
    return solution
```

4.3 3.3 Testing PI Controller (4 pts)

The following method can be invoked to test your implementation. The robot starts at the zero configuration, and we specify a list of target configurations at constant velocity.

[19]:
```
def test_pi(Kp, Ki):
    for i in range(num_joints):
        p.resetJointState(robot_id, i, 0, physicsClientId=sim_id)

    num_steps = 500
    q_start = np.zeros(num_joints)
    q_final = np.array([1.0, 0.8, -0.4, -1.2, 0.0, 0.5, -0.6])
```

```
    target_q = np.linspace(q_start, q_final, num_steps, endpoint=True)

    solution = pi_control(target_q, Kp, Ki)
    final_states = [p.getJointState(robot_id, i, sim_id) for i in␣
↪range(num_joints)]
    final_q_reached = np.array([s[0] for s in final_states])

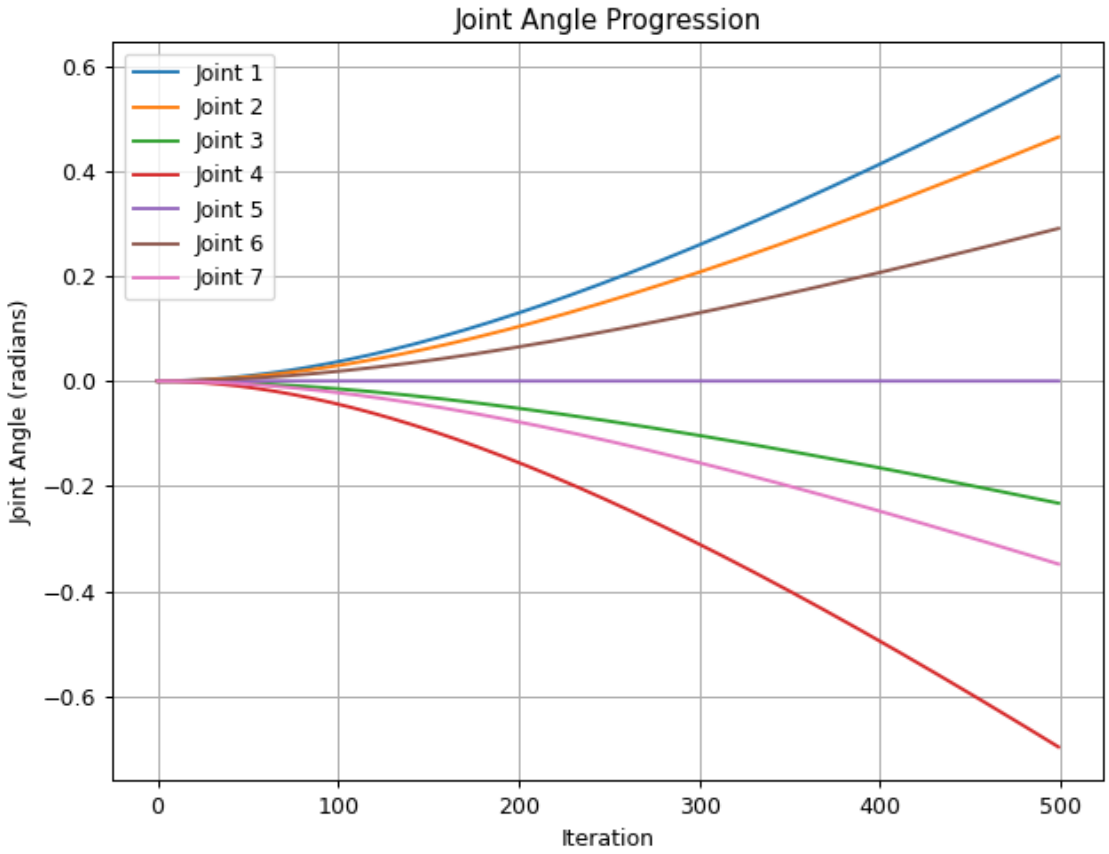
    print(f"Final q (target): {np.round(target_q[-1], 3)}")
    print(f"Final q (reached): {np.round(final_q_reached, 3)}")
    print(f"Final position error norm: {np.linalg.norm(target_q[-1] -␣
↪final_q_reached):.4f}")

    plot_joint_progression(solution)
    return solution
```

Run `test_pi()` twice, first with $K_p = 1$ and $K_i = 0$, and then with $K_p = 1$ and $K_i = 5$. Note that the former is just a P controller with no integral term. Show both plots. Briefly compare the two results and give an explanation for your observations, particularly on the differences in performance for this trajectory-tracking task.

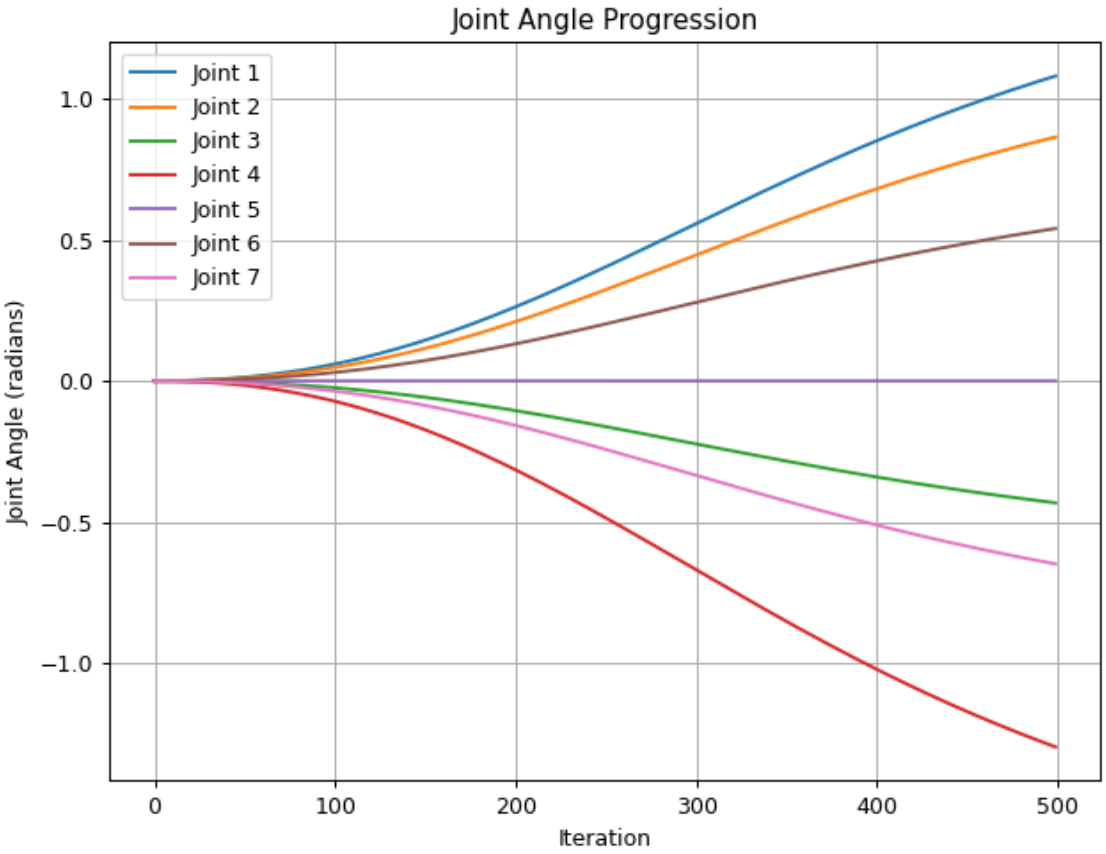
[20]:
```
solution_pi_p = test_pi(1.0, 0.0)
```

```
Final q (target): [ 1.   0.8 -0.4 -1.2  0.   0.5 -0.6]
Final q (reached): [ 0.581  0.465 -0.232 -0.697  0.     0.29  -0.348]
Final position error norm: 0.8228
```



[21]:
```
solution_pi = test_pi(1.0, 5.0)
```

```
Final q (target): [ 1.   0.8 -0.4 -1.2  0.   0.5 -0.6]
Final q (reached): [ 1.079  0.863 -0.431 -1.295  0.     0.539 -0.647]
Final position error norm: 0.1544
```



4.3.1 Answer to 3.3

For Kp = 1 and Ki = 0, the final error norm is about 0.8228 radians. A proportional controller needs a persistent position error to command the velocity required by the moving target, so the reached trajectory lags behind the reference. For Kp = 1 and Ki = 5, the final error norm drops to about 0.1544 radians. The integral accumulates this lag and supplies additional velocity even as the instantaneous error shrinks. This improves tracking, but the reached joints slightly overshoot the final target because accumulated integral action remains. The finite-duration test therefore shows reduced error, not perfect tracking.

4.4 References

1. Coumans, E., and Bai, Y. PyBullet Quickstart Guide. Bullet Physics SDK.
2. Bullet Physics SDK. PyBullet Jacobian example.
3. NumPy Developers. numpy.linalg.pinv. NumPy Reference Guide.
4. OpenAI. Codex. AI assistance: conversion and formatting.